

# EPAM SQL INTERVIEW

## Master Guide for SDET / Senior QA Automation / Test Lead

A practical interview workbook with a visual schema, 70+ questions, SQL coding problems, database concepts, test-data validation scenarios, window functions, joins, optimisation, and senior-level discussion points.

Important: No interview guide can guarantee every question asked at EPAM. The question bank below is designed around commonly reported EPAM SQL/database topics and the depth expected in senior SDET/QA automation interviews.

### How to use this PDF

1. Start with the visual Employee/Department schema.
2. Try every SQL problem yourself before reading the solution.
3. Use the topic sections as a revision checklist.
4. The PDF contains clickable bookmarks/TOC navigation for quick revision.

# Contents

1. EPAM SQL focus areas
2. Practice database: Employee + Department
3. SQL fundamentals
4. Filtering, sorting and NULL
5. GROUP BY, HAVING and aggregates
6. JOINS and self JOINS
7. Subqueries and set operations
8. Second/Nth highest and ranking
9. Window functions
10. CTEs and advanced queries
11. Duplicates and data quality
12. Dates, strings and CASE
13. DDL, DML, constraints and transactions
14. Indexes and query optimisation
15. Normalisation and database concepts
16. SDET database-testing scenarios
17. EPAM-style coding questions
18. Senior-level rapid-fire questions
19. Final revision checklist

Current interview context: Recent public reports describe EPAM technical interviews as broad, with SQL sometimes appearing alongside Java, Selenium, API testing and testing concepts. Reported SQL topics include joins, window functions, aggregate functions, set operations, ACID, normalisation and query-writing tasks. This guide therefore prioritises those areas.

# 1. EPAM SQL focus areas

| Area              | What to prepare                                      | Priority  |
|-------------------|------------------------------------------------------|-----------|
| SQL coding        | SELECT, WHERE, GROUP BY, HAVING, JOINS, subqueries   | Very high |
| Window functions  | ROW_NUMBER, RANK, DENSE_RANK, PARTITION BY, LAG/LEAD | Very high |
| Database concepts | Keys, constraints, normalisation, ACID, transactions | High      |
| Performance       | Indexes, execution plans, SARGability, query tuning  | High      |
| QA/SDET scenarios | API/UI to DB validation, data integrity, migrations  | Very high |
| Schema design     | Relationships and practical table design             | High      |

## 2. Practice database: Employee + Department

Use this same schema throughout the workbook. It intentionally contains duplicate salaries, manager relationships, NULL manager IDs and multiple departments so that interview questions can be realistic.

### Employee

| id  | name   | department_id | salary | manager_id | joining_date | email              |
|-----|--------|---------------|--------|------------|--------------|--------------------|
| 101 | Amit   | 10            | 150000 | NULL       | 2020-01-15   | amit@company.com   |
| 102 | Rahul  | 10            | 120000 | 101        | 2021-03-10   | rahul@company.com  |
| 103 | Priya  | 20            | 110000 | NULL       | 2019-06-20   | priya@company.com  |
| 104 | Sneha  | 20            | 90000  | 103        | 2022-02-15   | sneha@company.com  |
| 105 | Vikas  | 30            | 130000 | NULL       | 2020-08-12   | vikas@company.com  |
| 106 | Neha   | 30            | 90000  | 105        | 2023-01-10   | neha@company.com   |
| 107 | Rohit  | 10            | 120000 | 101        | 2022-05-18   | rohit@company.com  |
| 108 | Pooja  | 20            | 80000  | 103        | 2023-07-25   | pooja@company.com  |
| 109 | Karan  | 40            | 70000  | NULL       | 2021-11-05   | karan@company.com  |
| 110 | Anjali | 40            | 60000  | 109        | 2024-01-15   | anjali@company.com |
| 111 | Suresh | 50            | 100000 | NULL       | 2020-09-10   | suresh@company.com |
| 112 | Meena  | 50            | 85000  | 111        | 2022-10-20   | meena@company.com  |
| 113 | Arjun  | 10            | 150000 | 101        | 2024-02-15   | arjun@company.com  |
| 114 | Kavita | 20            | 90000  | 103        | 2024-05-10   | kavita@company.com |
| 115 | Ramesh | 30            | 130000 | 105        | 2025-01-15   | ramesh@company.com |

### Department

| department_id | department_name |
|---------------|-----------------|
| 10            | Engineering     |
| 20            | QA              |
| 30            | DevOps          |
| 40            | HR              |
| 50            | Finance         |

Relationships: Employee.department\_id → Department.department\_id. Employee.manager\_id → Employee.id (self-referencing relationship).

## 3. SQL fundamentals

### Q1. What is SQL?

Answer: SQL (Structured Query Language) is used to define, query, manipulate and control data in relational databases.

### Q2. What is an RDBMS?

Answer: A relational database management system stores data in related tables and typically enforces relationships using keys and constraints.

### Q3. What is a primary key?

Answer: A column or combination of columns that uniquely identifies each row. It cannot contain NULL.

### Q4. What is a foreign key?

Answer: A column that references a key in another table and helps maintain referential integrity.

### Q5. What is a candidate key?

Answer: A minimal set of columns that can uniquely identify a row; one candidate key can be selected as the primary key.

### Q6. What is a composite key?

Answer: A key made from two or more columns.

### Q7. What is a UNIQUE constraint?

Answer: It prevents duplicate values in a column or column combination; NULL behaviour depends on the database.

### Q8. What is the difference between CHAR and VARCHAR?

Answer: CHAR is fixed-length; VARCHAR is variable-length. Exact storage and performance details vary by database engine.

### Q9. What is NULL?

Answer: NULL represents an unknown or missing value; it is not the same as zero or an empty string.

## 4. Filtering, sorting and NULL

### Q10. Select all employees.

```
SELECT *  
FROM Employee;
```

### Q11. Find employees earning more than 100000.

```
SELECT *  
FROM Employee  
WHERE salary > 100000;
```

### Q12. Find employees whose name starts with A.

```
SELECT *  
FROM Employee  
WHERE name LIKE 'A%';
```

### Q13. Find employees whose salary is between 80000 and 120000.

```
SELECT *  
FROM Employee  
WHERE salary BETWEEN 80000 AND 120000;
```

### Q14. Find employees in departments 10, 20 or 30.

```
SELECT *  
FROM Employee  
WHERE department_id IN (10, 20, 30);
```

### Q15. Find employees who do not have a manager.

```
SELECT *  
FROM Employee  
WHERE manager_id IS NULL;
```

### Q16. Why is `manager\_id = NULL` incorrect?

Answer: NULL must be tested using IS NULL or IS NOT NULL because comparisons with NULL evaluate to UNKNOWN.

### Q17. Sort employees by salary from highest to lowest.

```
SELECT *  
FROM Employee  
ORDER BY salary DESC;
```

### Q18. Select unique salaries.

```
SELECT DISTINCT salary  
FROM Employee  
ORDER BY salary DESC;
```

## 5. GROUP BY, HAVING and aggregates

### Q19. Find the number of employees.

```
SELECT COUNT(*) AS employee_count
FROM Employee;
```

### Q20. Find the highest, lowest and average salary.

```
SELECT MAX(salary) AS max_salary,
       MIN(salary) AS min_salary,
       AVG(salary) AS avg_salary
FROM Employee;
```

### Q21. Find the number of employees in each department.

```
SELECT department_id,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department_id;
```

### Q22. Find departments having more than two employees.

```
SELECT department_id,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department_id
HAVING COUNT(*) > 2;
```

### Q23. Explain WHERE vs HAVING.

Answer: WHERE filters individual rows before grouping; HAVING filters grouped results after GROUP BY.

### Q24. Find the average salary for each department.

```
SELECT department_id,
       AVG(salary) AS avg_salary
FROM Employee
GROUP BY department_id;
```

### Q25. Find the total salary paid by each department.

```
SELECT department_id,
       SUM(salary) AS total_salary
FROM Employee
GROUP BY department_id;
```

## 6. JOINS and self JOINS

### Q26. What is an INNER JOIN?

Answer: It returns rows where the join condition matches in both tables.

### Q27. What is a LEFT JOIN?

Answer: It returns all rows from the left table and matching rows from the right table; unmatched right-side columns are NULL.

### Q28. Show employee name and department name.

```
SELECT e.name,  
       d.department_name  
FROM Employee e  
INNER JOIN Department d  
    ON e.department_id = d.department_id;
```

### Q29. Find employees whose department does not exist.

```
SELECT e.*  
FROM Employee e  
LEFT JOIN Department d  
    ON e.department_id = d.department_id  
WHERE d.department_id IS NULL;
```

### Q30. Find departments with no employees.

```
SELECT d.*  
FROM Department d  
LEFT JOIN Employee e  
    ON d.department_id = e.department_id  
WHERE e.id IS NULL;
```

### Q31. What is a self JOIN?

Answer: A self JOIN joins a table to itself. It is useful for hierarchical data such as employee-manager relationships.

### Q32. Show employee and manager names.

```
SELECT e.name AS employee,  
       m.name AS manager  
FROM Employee e  
LEFT JOIN Employee m  
    ON e.manager_id = m.id;
```

### Q33. Find employees earning more than their manager.

```
SELECT e.name AS employee,  
       e.salary AS employee_salary,  
       m.name AS manager,  
       m.salary AS manager_salary  
FROM Employee e  
JOIN Employee m  
    ON e.manager_id = m.id  
WHERE e.salary > m.salary;
```

### Q34. Explain CROSS JOIN.

Answer: It returns the Cartesian product: every row from one table paired with every row from the other table.

### Q35. What is a FULL OUTER JOIN?

Answer: It returns matching rows plus unmatched rows from both sides. Support varies by database engine.

## 7. Subqueries and set operations

### Q36. Find employees earning more than the company average.

```
SELECT *  
FROM Employee  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM Employee  
);
```

### Q37. What is a correlated subquery?

Answer: A subquery that references a column from the outer query, so it can be evaluated in relation to each outer row.

### Q38. What is EXISTS?

Answer: EXISTS checks whether the subquery returns at least one row; it is often useful for existence checks.

### Q39. Show employees whose department exists in Department.

```
SELECT *  
FROM Employee e  
WHERE EXISTS (  
    SELECT 1  
    FROM Department d  
    WHERE d.department_id = e.department_id  
);
```

### Q40. UNION vs UNION ALL?

Answer: UNION combines result sets and removes duplicates; UNION ALL combines them without duplicate elimination and is usually cheaper.

### Q41. What requirements must UNION have?

Answer: The SELECT statements must return the same number of compatible columns in corresponding positions.

### Q42. Find employees whose salary is above the Engineering average.

```
SELECT *  
FROM Employee  
WHERE salary > (  
    SELECT AVG(e2.salary)  
    FROM Employee e2  
    WHERE e2.department_id = 10  
);
```

## 8. Second/Nth highest and ranking

### Q43. Find the highest salary.

```
SELECT MAX(salary) AS highest_salary
FROM Employee;
```

### Q44. Find the second-highest distinct salary.

```
SELECT MAX(salary) AS second_highest
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
);
```

### Q45. Find the third-highest distinct salary using DENSE\_RANK.

```
SELECT name, salary
FROM (
    SELECT name, salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM Employee
) x
WHERE rnk = 3;
```

### Q46. Why can OFFSET/LIMIT be risky for Nth-highest salary?

Answer: If duplicate salaries exist, simple row offsets may return a different row rather than the Nth distinct salary. DENSE\_RANK is clearer when distinct salary ranks are required.

### Q47. Find employees with the highest salary in each department.

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM Employee e
) x
WHERE rnk = 1;
```

### Q48. Find the top 3 salaries in each department.

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM Employee e
) x
WHERE rnk <= 3;
```

## 9. Window functions

### Q49. What is a window function?

Answer: A window function calculates across related rows while retaining the individual rows in the result.

### Q50. ROW\_NUMBER vs RANK vs DENSE\_RANK?

Answer: ROW\_NUMBER gives every row a unique sequence. RANK gives tied rows the same rank and leaves gaps. DENSE\_RANK gives tied rows the same rank without gaps.

### Q51. Generate a salary row number.

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM Employee;
```

### Q52. Rank salaries within each department.

```
SELECT name, department_id, salary,
       RANK() OVER (
         PARTITION BY department_id
         ORDER BY salary DESC
       ) AS salary_rank
FROM Employee;
```

### Q53. What does PARTITION BY do?

Answer: It divides rows into independent groups for the window calculation without collapsing them like GROUP BY does.

### Q54. Find each employee's salary difference from their department average.

```
SELECT name, department_id, salary,
       salary - AVG(salary) OVER (
         PARTITION BY department_id
       ) AS difference_from_dept_avg
FROM Employee;
```

### Q55. Use LAG to show the previous salary in descending order.

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary DESC) AS previous_salary
FROM Employee;
```

### Q56. Use LEAD to show the next salary.

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary DESC) AS next_salary
FROM Employee;
```

### Q57. Calculate a running salary total ordered by employee ID.

```
SELECT id, name, salary,
       SUM(salary) OVER (
         ORDER BY id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM Employee;
```

## 10. CTEs and advanced queries

### Q58. What is a CTE?

Answer: A Common Table Expression is a named temporary result defined with WITH and used by the following SQL statement.

### Q59. Find the top 2 salaries per department using a CTE.

```
WITH ranked AS (  
    SELECT e.*,  
           DENSE_RANK() OVER (  
               PARTITION BY department_id  
               ORDER BY salary DESC  
           ) AS rnk  
    FROM Employee e  
)  
SELECT *  
FROM ranked  
WHERE rnk <= 2;
```

### Q60. CTE vs subquery?

Answer: Both can express intermediate results. CTEs can improve readability and can be reused within a statement; optimisation/materialisation behaviour depends on the database.

### Q61. What is a recursive CTE?

Answer: A CTE that references itself, commonly used for hierarchies such as employee-manager trees. Exact syntax varies by database.

### Q62. How would you find an employee's full management chain?

Answer: Use a recursive CTE starting from the employee and repeatedly joining manager\_id to id until the top-level manager is reached.

# 11. Duplicates and data quality

## Q63. Find duplicate salaries.

```
SELECT salary, COUNT(*) AS cnt
FROM Employee
GROUP BY salary
HAVING COUNT(*) > 1;
```

## Q64. Find duplicate email addresses.

```
SELECT email, COUNT(*) AS cnt
FROM Employee
GROUP BY email
HAVING COUNT(*) > 1;
```

## Q65. How do you identify duplicate rows when there is no unique ID?

Answer: Group by the business-key columns and use HAVING COUNT(\*) > 1, or use ROW\_NUMBER partitioned by those columns.

## Q66. Find duplicate records using ROW\_NUMBER.

```
SELECT *
FROM (
    SELECT e.*,
           ROW_NUMBER() OVER (
               PARTITION BY name, department_id, salary, email
               ORDER BY id
           ) AS rn
    FROM Employee e
) x
WHERE rn > 1;
```

## Q67. How would you delete duplicates safely?

Answer: First identify the exact duplicate business key and decide which row to retain. Use a transaction, verify the rows to be deleted with SELECT, then delete only rows whose ROW\_NUMBER is greater than 1. Exact DELETE syntax varies by database.

## Q68. What database checks would you automate after a migration?

Answer: Row counts, nullability, uniqueness, foreign-key integrity, data type mapping, transformed values, boundary values, timestamps, aggregates and representative business scenarios.

## 12. Dates, strings and CASE

### Q69. Find employees who joined in 2024.

```
SELECT *
FROM Employee
WHERE joining_date >= '2024-01-01'
AND joining_date < '2025-01-01';
```

### Q70. Find employees whose name contains 'a'.

```
SELECT *
FROM Employee
WHERE name LIKE '%a%';
```

### Q71. Create a salary category.

```
SELECT name, salary,
CASE
    WHEN salary >= 130000 THEN 'HIGH'
    WHEN salary >= 90000 THEN 'MEDIUM'
    ELSE 'LOW'
END AS salary_category
FROM Employee;
```

### Q72. Why should date filtering often use a half-open range?

Answer: A range such as  $\geq$  start and  $<$  next\_day/month/year avoids time-of-day edge cases when a timestamp column contains hours, minutes and seconds.

### Q73. What is COALESCE?

Answer: It returns the first non-NULL expression. Example: COALESCE(manager\_id, 0).

### Q74. Replace NULL manager IDs with 'Top Level'.

```
SELECT name,
COALESCE(CAST(manager_id AS VARCHAR), 'Top Level') AS manager
FROM Employee;
```

## 13. DDL, DML, constraints and transactions

### Q75. DDL vs DML?

Answer: DDL defines database objects (CREATE, ALTER, DROP, TRUNCATE in common usage). DML changes data (INSERT, UPDATE, DELETE). Exact classification can vary by engine.

### Q76. DELETE vs TRUNCATE vs DROP?

Answer: DELETE removes rows and can use a WHERE clause. TRUNCATE removes all rows and is generally a bulk operation. DROP removes the table object itself. Transaction and logging behaviour varies by database.

### Q77. What is a transaction?

Answer: A transaction is a logical unit of work whose changes are committed or rolled back according to the database's transaction semantics.

### Q78. Explain ACID.

Answer: Atomicity: all-or-nothing. Consistency: constraints/invariants remain valid. Isolation: concurrent transactions are controlled according to the isolation level. Durability: committed changes survive failures.

### Q79. What is COMMIT?

Answer: Makes the transaction's changes permanent according to the database's transaction semantics.

### Q80. What is ROLLBACK?

Answer: Reverts uncommitted changes in the current transaction.

### Q81. What is a SAVEPOINT?

Answer: A marker inside a transaction to which you can roll back without necessarily rolling back the entire transaction.

### Q82. What is a deadlock?

Answer: Two or more transactions wait on resources held by each other, creating a cycle. Databases typically detect and abort one transaction.

### Q83. What is isolation level?

Answer: A database setting defining which effects of concurrent transactions are visible. Common levels include Read Uncommitted, Read Committed, Repeatable Read and Serializable.

## 14. Indexes and query optimisation

### Q84. What is an index?

Answer: A data structure that can speed up data retrieval for suitable predicates and ordering, at the cost of storage and write/update overhead.

### Q85. Clustered vs non-clustered index?

Answer: A clustered index determines the physical/logical organisation of table rows in engines that support that concept; a non-clustered index is a separate access structure. Exact implementation differs by database.

### Q86. What columns are good index candidates?

Answer: Columns frequently used in selective WHERE predicates, JOIN conditions, ORDER BY/GROUP BY patterns and unique lookups, based on actual workload and execution plans.

### Q87. Why can too many indexes be bad?

Answer: They consume storage and increase the work required for INSERT/UPDATE/DELETE operations. They can also add maintenance complexity.

### Q88. What is an execution plan?

Answer: The database's chosen strategy for executing a query, including scans, index access, joins, sorts and estimated costs.

### Q89. How would you troubleshoot a slow SQL query?

Answer: Reproduce it with representative data, inspect the execution plan, check indexes and predicates, verify statistics, reduce unnecessary rows/columns, review joins and sorting, and measure again after each change.

### Q90. What is a full table scan?

Answer: The database reads many or all rows in the table instead of using a more selective access path. A scan is not inherently bad; it can be appropriate for small tables or large result sets.

### Q91. What is SARGability?

Answer: A predicate is SARGable when the database can efficiently use an index/access path for it. Wrapping indexed columns in functions can prevent efficient index use in some engines.

## 15. Normalisation and database concepts

### Q92. What is normalisation?

Answer: A process of structuring relational data to reduce unnecessary duplication and update anomalies.

### Q93. What is 1NF?

Answer: Values are atomic and repeating groups are removed; each row/column intersection represents a single value.

### Q94. What is 2NF?

Answer: 1NF plus every non-key attribute depends on the whole candidate key, relevant mainly when a composite key exists.

### Q95. What is 3NF?

Answer: 2NF plus non-key attributes should not depend transitively on another non-key attribute.

### Q96. What is denormalisation?

Answer: Intentionally introducing redundancy to improve read performance or simplify access, while accepting extra consistency/storage considerations.

### Q97. What is referential integrity?

Answer: A foreign-key relationship prevents child rows from referencing nonexistent parent keys, subject to the database's constraint rules.

### Q98. Primary key vs unique key?

Answer: Both enforce uniqueness, but a primary key is the table's chosen main identifier and cannot be NULL; UNIQUE constraint NULL semantics vary by database.

## 16. SDET database-testing scenarios

### Q99. How do you validate that a UI-created record reached the database?

Answer: Capture a stable business identifier from the UI/API, query the database using that identifier, and compare expected versus actual values. Avoid relying on brittle row ordering.

### Q100. How do you validate an API response against DB data?

Answer: Send the API request, extract the entity ID/business key, query the relevant DB rows, normalise formatting/time zones where appropriate, then assert the required fields and business rules.

### Q101. How would you validate a DELETE API?

Answer: Verify the API status/response, then query the DB to confirm the expected deletion or soft-delete flag, depending on the product design.

### Q102. How would you validate a soft delete?

Answer: Query by the entity ID and verify the record remains present with the expected deletion marker such as `is_deleted = true`, plus any audit fields.

### Q103. How do you validate a database migration?

Answer: Compare schema before/after, verify row counts, constraints and indexes, validate transformed columns and sampled records, test null/default behaviour, and run business-critical queries.

### Q104. How would you detect orphan records?

Answer: Use a LEFT JOIN from the child table to the parent and filter where the parent's key is NULL, or enforce a foreign key if appropriate.

### Q105. How would you test transaction rollback?

Answer: Perform an operation that intentionally fails after an earlier change, then verify that the earlier change was not committed.

### Q106. What SQL checks are useful in regression automation?

Answer: Counts, uniqueness, null checks, referential integrity, status transitions, financial totals, timestamps, audit records and cross-table consistency checks.

### Q107. What should you avoid in DB assertions?

Answer: Avoid hardcoding volatile IDs, relying on row order without ORDER BY, exposing credentials, modifying production data, and asserting implementation details that are not part of the contract.

## 17. EPAM-style coding questions

### Q108. Find the second-highest salary.

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

### Q109. Find the top 3 salaries in each department.

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id ORDER BY salary DESC
           ) AS rn
    FROM Employee e
) x
WHERE rn <= 3;
```

### Q110. Find employees earning more than their manager.

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS manager_salary
FROM Employee e
JOIN Employee m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

### Q111. Find the department with the highest average salary.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY department_id
ORDER BY avg_salary DESC
FETCH FIRST 1 ROW ONLY;
```

### Q112. Find departments with more than 3 employees.

```
SELECT department_id, COUNT(*) AS cnt
FROM Employee
GROUP BY department_id
HAVING COUNT(*) > 3;
```

### Q113. Find employees who joined after their manager.

```
SELECT e.name AS employee, e.joining_date,
       m.name AS manager, m.joining_date AS manager_joining_date
FROM Employee e
JOIN Employee m ON e.manager_id = m.id
WHERE e.joining_date > m.joining_date;
```

### Q114. Find each department's highest and lowest salary.

```
SELECT department_id,
       MAX(salary) AS max_salary,
       MIN(salary) AS min_salary
FROM Employee
GROUP BY department_id;
```

### Q115. Find salary duplicates and list the employees.

```
SELECT e.*
FROM Employee e
JOIN (
    SELECT salary
    FROM Employee
    GROUP BY salary
    HAVING COUNT(*) > 1
) d ON e.salary = d.salary
ORDER BY e.salary DESC, e.id;
```

### Q116. Find employees whose salary is above their department average.

```
SELECT *
FROM Employee e
WHERE salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department_id = e.department_id
);
```

**Q117. Show employee count and average salary by department name.**

```
SELECT d.department_name,
       COUNT(e.id) AS employee_count,
       AVG(e.salary) AS avg_salary
FROM Department d
LEFT JOIN Employee e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
```

## 18. Senior-level rapid-fire questions

### Q118. WHERE vs HAVING?

Answer: WHERE filters rows; HAVING filters groups.

### Q119. INNER JOIN vs LEFT JOIN?

Answer: INNER keeps matches; LEFT keeps all left rows and matching right rows.

### Q120. GROUP BY vs PARTITION BY?

Answer: GROUP BY collapses rows into groups; PARTITION BY keeps rows and defines windows for calculations.

### Q121. RANK vs DENSE\_RANK?

Answer: RANK leaves gaps after ties; DENSE\_RANK does not.

### Q122. COUNT(\*) vs COUNT(column)?

Answer: COUNT(\*) counts rows; COUNT(column) ignores NULLs in that column.

### Q123. UNION vs UNION ALL?

Answer: UNION removes duplicates; UNION ALL does not.

### Q124. EXISTS vs IN?

Answer: Both can express membership/existence; EXISTS is especially natural for correlated existence checks. Optimisation depends on the database and data.

### Q125. Primary key vs foreign key?

Answer: Primary key identifies a row; foreign key references a key in another table.

### Q126. DELETE vs TRUNCATE?

Answer: DELETE removes rows and can filter; TRUNCATE removes all rows as a bulk operation, with engine-specific transaction/logging semantics.

### Q127. Index benefit and cost?

Answer: Faster suitable reads; additional storage and write/maintenance cost.

### Q128. What is a deadlock?

Answer: A cycle of transactions waiting for each other's locks/resources.

### Q129. What is a stored procedure?

Answer: A database-side program containing SQL and procedural logic; capabilities vary by engine.

### Q130. What is a view?

Answer: A saved query presented as a virtual table; materialisation behaviour depends on whether it is a normal or materialized view.

### Q131. What is a trigger?

Answer: Database logic that executes automatically in response to specified events such as INSERT/UPDATE/DELETE.

### Q132. What is a composite index?

Answer: An index over multiple columns; column order matters for many access patterns.

## 19. Final revision checklist

- ☐ 1. Can I write SELECT, WHERE, ORDER BY, DISTINCT and LIKE without hesitation?
- ☐ 2. Can I explain NULL and use IS NULL / IS NOT NULL correctly?
- ☐ 3. Can I write INNER, LEFT and SELF JOIN queries?
- ☐ 4. Can I explain GROUP BY vs HAVING?
- ☐ 5. Can I solve second-highest and Nth-highest salary questions?
- ☐ 6. Can I use ROW\_NUMBER, RANK and DENSE\_RANK?
- ☐ 7. Can I solve Top-N-per-group problems?
- ☐ 8. Can I write correlated subqueries and EXISTS?
- ☐ 9. Can I explain UNION vs UNION ALL?
- ☐ 10. Can I identify and remove duplicates safely?
- ☐ 11. Can I explain CTEs and recursive CTEs?
- ☐ 12. Can I explain primary key, foreign key, unique, NOT NULL and CHECK constraints?
- ☐ 13. Can I explain ACID and transaction isolation?
- ☐ 14. Can I explain indexes and execution plans?
- ☐ 15. Can I troubleshoot a slow query?
- ☐ 16. Can I validate UI/API data against the database as an SDET?
- ☐ 17. Can I test database migrations and data integrity?
- ☐ 18. Can I explain my SQL query line-by-line to an interviewer?

Final interview strategy: For a senior SDET role, do not only give the query. Explain the data assumption, why you selected the JOIN/subquery/window function, how NULLs and duplicates are handled, the expected complexity/performance considerations, and how you would turn the check into an automated DB assertion.

Source note: The guide is a preparation resource, not an official EPAM question paper. Recent public interview reports were used to shape the topic emphasis; individual interview loops vary by role, team, location and interviewer.